

AI-Powered Credit Risk Intelligence Platform

NeoStats AI Engineer Assignment — Submission

- Dataset: Home Credit Default Risk (Kaggle)
- Stack: Python 3.11, LightGBM, SHAP, Streamlit, SQLite, Docker
- LLM: Multi-provider (OpenAI / Groq / Gemini) with deterministic fallback

Business Problem

- Banks must make faster, more accurate, and explainable credit decisions.
- Identify high-risk applicants early; automate risk scoring.
- Provide auditable, regulator-friendly reasons for every decision.
- Let business analysts explore the portfolio in plain English.
- Bridge ML insights and credit policy through readable rules.

Solution at a Glance

- End-to-end platform: EDA → ML → Explainability → Rules → Chatbot.
- Single Streamlit UI with 6 sections; one Docker command to run.
- ML model returns probability + Low/Medium/High band + Decision.
- SHAP TreeExplainer attributes every score to ranked feature drivers.
- Talk-to-Data agent translates English to safe, read-only SQLite SQL.
- Decision-tree rules give credit officers an audit-ready policy view.

Architecture

- Streamlit UI → src/ml (LightGBM + SHAP) and src/rules (decision tree).
- src/data ingests CSV → SQLite (single source of truth for ML & chatbot).
- src/llm: provider auto-detect (OpenAI → Groq → Gemini → fallback parser).
- src/utis/sql_safety: blocks DDL/DML, single-statement, LIMIT enforced.
- Dockerfile pre-trains the model at build time so the app boots ready.

Module 1 — EDA Insights

- EXT_SOURCE_* are the strongest single predictors; high scores cut default ~5×.
- Credit-to-income ratio matters more than absolute income.
- Higher-education / Academic-degree applicants default less.
- Younger applicants (≤ 30 y) are riskier; flattens after ~45.
- Employment tenure protects; sentinel DAYS_EMPLOYED=365243 must be NaN.

Module 2 — Talk-to-Data Chatbot

- Pipeline: NL question → LLM → JSON {sql} → safety guard → run → summarise.
- Pinned schema + 4 few-shot examples + temperature=0 + max 256 output tokens.
- Static guardrails: SELECT-only, single statement, LIMIT cap, table allowlist.
- Read-only SQLite connection (mode=ro) for execution.
- Deterministic fallback parser handles 9 canonical questions offline.

Module 3 — Machine Learning Layer

- Algorithm: LightGBM • scale_pos_weight handles 8% imbalance.
- Validation: 80/20 stratified split.
- ROC-AUC: 0.8952 | KS: 0.6571 | PR-AUC: 0.5109
- Operating threshold: 0.0631 (Youden's J).
- Risk bands: Low <0.20, Medium <0.50, High otherwise.
- Decision: Approve / Review / Reject (configurable in .env).

Module 4 — Explainable AI (SHAP)

- Per-applicant: top contributors with sign and direction (increases / decreases risk).
- Global: mean |SHAP| over a sample of 400 rows.
- Exact Shapley values via TreeExplainer — no approximation.
- EXT_SOURCE_MEAN, CREDIT_INCOME_RATIO, EXT_SOURCE_2 dominate the global ranking.

Module 5 — Decision Rules

- Depth-4 tree fit on the same engineered features as the model.
- Each leaf → IF-THEN rule with support, default-rate, and lift.
- Top rules in the bundled run:
 - R1: High | support 11.79% | default rate 66.72% | lift 8.34×
 - R13: High | support 6.86% | default rate 88.31% | lift 11.04×
 - R8: Medium | support 17.3% | default rate 24.71% | lift 3.09×
 - R7: High | support 5.79% | default rate 55.18% | lift 6.9×
 - R9: High | support 2.68% | default rate 79.95% | lift 9.99×

Module 6 — Dockerised Deployment

- One command: `docker compose up --build`.
- Multi-stage Dockerfile (builder + slim runtime, non-root user).
- Pre-trains the model at build time; healthcheck on `/_stcore/health`.
- Volumes: `data/raw` is read-only mount; `models/` persisted on host.
- `.env.example` documents every variable; auto-detect picks any LLM key.

Prompt Engineering & Hallucination Control

- JSON-only contract: model emits {"sql": "..."}; we parse deterministically.
- Pinned schema with column descriptions reduces guessing.
- Few-shot examples are short (4 cases) to keep input tokens low.
- Temperature 0; max 256 output tokens.
- Static SQL safety: regex + sqlparse validates before execution.
- Result-grounded summary prompt forbids invented numbers.
- Hard fallback path keeps the demo working when the LLM is unavailable.

Token Optimisation

- Schema (~25 columns) sent once per turn; no DB introspection round-trip.
- Compact column descriptions (1 line each) instead of verbose comments.
- Few-shot examples reuse the same column names so the model latches faster.
- max_tokens caps output; we expect <50 tokens for most SQL replies.
- Summarisation only sees the result rows we actually return (≤ 20).

Evaluation Results

Metric	Value
ROC-AUC	0.8952
PR-AUC	0.5109
KS Statistic	0.6571
F1 @ optimal threshold	0.3994
Optimal threshold	0.0631
Default rate	0.08
Train rows / Val rows	8000 / 2000

How To Run

- `git clone https://github.com/ashwani-dhayal/credit-risk-platform.git`
- `cd credit-risk-platform`
- `cp .env.example .env # add ONE LLM key (optional)`
- `docker compose up --build`
- Open `http://localhost:8501`
- Local mode: `python -m venv .venv && pip install -r requirements.txt; python scripts/train_model.py; streamlit run app/streamlit_app.py`

Limitations & Next Steps

- Only application_train modelled — adding bureau / previous_application feature aggregations should lift AUC by ~0.03 on real Kaggle data.
- No data-drift monitoring yet (Evidently / WhyLogs are obvious next steps).
- Single-tenant SQLite — switch to Postgres for multi-user.
- No UI auth — put behind a reverse proxy in production.
- LLM cost cap is per-call; daily quotas would need Redis-based rate limiting.

Thank you

- Repo: <https://github.com/ashwani-dhayal/credit-risk-platform>
- Author: Ashwani Dhayal
- Submission for NeoStats AI Engineer assignment.